CSE333 LAB
Threads

Zuhal Altuntaş

[Slides by Dr. Sanem Arslan Yılmaz.
Adopted from UNIX SYSTEMS Programming: Communication, Concurrency, and Threads, and
https://computing.llnl.gov/tutorials/pthreads/]

What is a Thread?

What is a Thread?

- Technically, a thread is defined as an independent stream of instructions that can be scheduled to run as such by the operating system.

What is a Thread?

- Technically, a thread is defined as an independent stream of instructions that can be scheduled to run as such by the operating system.
- To the software developer, the concept of a "procedure" that runs independently from its main program may best describe a thread.

What is a Thread? (cont.)

- Threads use and exist within these process resources.
- It dies if the process dies.
- It is "lightweight" because most of the overhead has already been accomplished through the creation of its process.

Thread Package

Thread Package

- A thread package usually includes functions for thread creation and thread destruction, scheduling, and enforcement of mutual exclusion
- When a thread is created, the runtime system allocates data structures to hold the thread's ID, registers, stack, and program counter value

Thread Package

- A thread package usually includes functions for thread creation and thread destruction, scheduling, and enforcement of mutual exclusion
- When a thread is created, the runtime system allocates data structures to hold the thread's ID, registers, stack, and program counter value
- The threads for a process share the entire address space of that process
- They can modify global variables, access open file descriptors, and cooperate and interfere with each other in many ways

Thread Package

- A thread package usually includes functions for thread creation and thread destruction, scheduling, and enforcement of mutual exclusion
- When a thread is created, the runtime system allocates data structures to hold the thread's ID, registers, stack, and program counter value
- The threads for a process share the entire address space of that process
- They can modify global variables, access open file descriptors, and cooperate and interfere with each other in many ways
- POSIX threads are often called pthreads because all of the thread functions start with pthread
- POSIX threads are referenced by an ID of type `pthread_t`
- Most POSIX functions return zero if successful; otherwise, they return a nonzero error

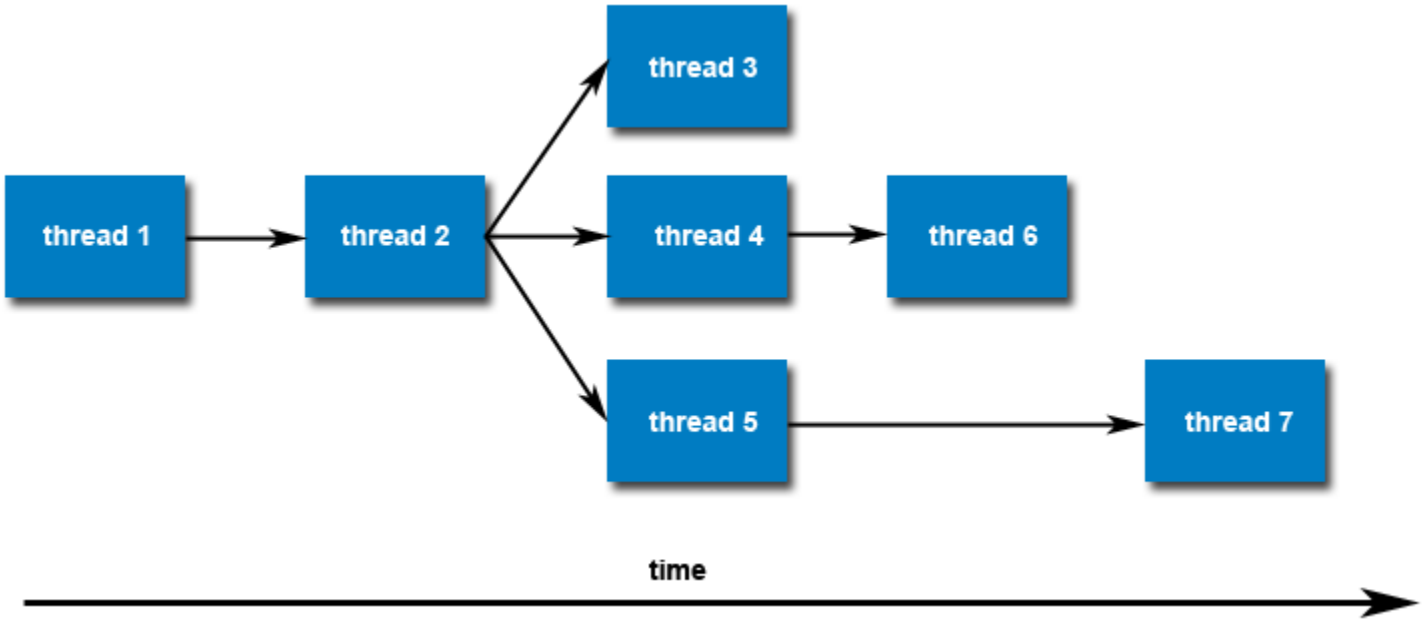
Creating Threads

Initially, your main() program comprises a single, default thread.

-
- All other threads must be explicitly created by the programmer.
-
- pthread_create creates a new thread and makes it executable.

Creating Threads (cont.)

- Once created, threads are peers, and may create other threads. There is no implied hierarchy or dependency between threads.



Creating Threads (cont.)

```
pthread_create (pthread_t *thread,
        const pthread_attr_t *attributes,
        void *(*start_routine(void *),
          void *argument)
```

- threadid: A unique identifier for the new thread.
  -
  - attr: An attribute object that may be used to set thread attributes. You can specify a thread attributes object, or NULL for the default values.
  -
  - start_routine: the C function that the thread will execute once it is created.
    - The function must take a single parameter of type `void *`
    - The function must return a pointer of type `void *`, which is treated as an exit status.
    -
  - arg: A single argument that may be passed to *start_routine*. It must be passed by reference as a pointer cast of type void. NULL may be used if no argument is to be passed.
    - To pass multiple values as a parameter to a thread, a pointer to an array or structure can be used

Terminating Threads

- There are several ways in which a thread may be terminated:
  - The thread returns normally from its starting routine. It's work is done.
  - The thread makes a call to the pthread_exit subroutine whether its work is done or not.
    - `void pthread_exit(void *valuePtr);`
    -
  - The thread is canceled by another thread via the pthread_cancel routine.
    - `void pthread_cancel(pthread_t targetThread);`
    -
  - The entire process is terminated due to making a call to either the exec() or exit().

Terminating Threads

- If main() finishes before the threads it has created, and exits with pthread_exit , the other threads will continue to execute.
- Otherwise, they will be automatically terminated when main() finishes.
- The pthread_exit() routine allows the programmer to specify an optional termination *status* parameter.

Examples

1. thread_create.c

Examples

1. thread_create.c

How to Compile:
gcc thread_create.c –o thread_create.out -lpthread

How to Run:
./thread_create.out

Examples

1. thread_create.c

How to Compile:
gcc thread_create.c –o thread_create.out -lpthread

How to Run:
./thread_create.out

2. hello.c

Passing Arguments to Threads

- The pthread_create() routine permits the programmer to pass one argument to the thread start routine.
- For cases where multiple arguments must be passed, this limitation is easily overcome by creating a structure which contains all of the arguments, and then passing a pointer to that structure in the pthread_create() routine.
- All arguments must be passed by reference and cast to (void *).

Examples

1. thread_create2.c

Examples

1. thread_create2.c

How to Compile:

gcc thread_create2.c –o thread_create2.out -lpthread

How to Run:

./thread_create2.out

Examples

1. thread_create2.c

How to Compile:

gcc thread_create2.c –o thread_create2.out -lpthread

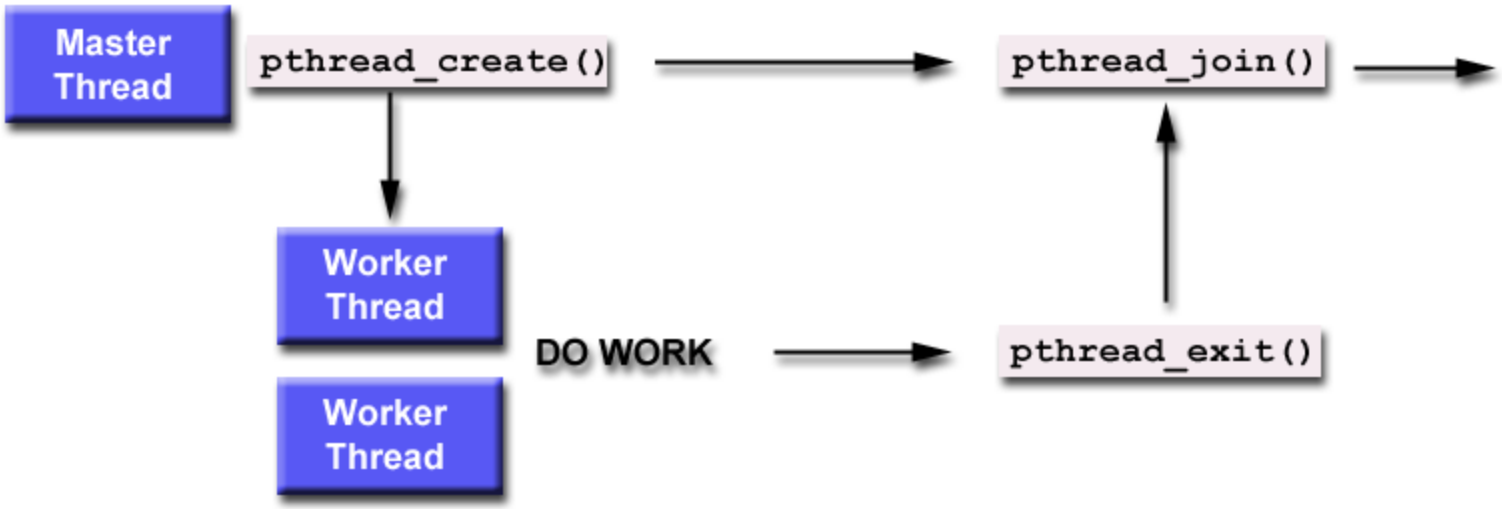
How to Run:

./thread_create2.out

2. hello_arg2.c

Joining Threads

- "Joining" is one way to accomplish synchronization between threads



Joining Threads (cont.)

pthread_join (`pthread_t` **threadid,**

- `void **` **status)**
- The pthread_join() subroutine blocks the calling thread until the specified threadid thread terminates.
- The programmer is able to obtain the target thread's termination return status if it was specified in the target thread's call to pthread_exit().

- A joining thread can match one pthread_join() call.

Examples

1. thread_join.c

Examples

1. thread_join.c

How to Compile:
gcc thread_join.c –o thread_join.out -lpthread

How to Run:
./thread_join.out

Examples

1. thread_join.c

How to Compile:
gcc thread_join.c –o thread_join.out -lpthread

How to Run:
./thread_join.out

2. join.c

Thread Management

- **pthread_self ()**
  - A thread can find out its ID by calling `pthread_self()`
  - The function returns the thread ID of the calling thread

- **pthread_equal** (`pthread_t threadA, pthread_t threadB`)
  - compares two thread IDs. If the two IDs are different 0 is returned, otherwise a non-zero value is returned.

Ex:

```
pthread_t currentThread;

if (pthread_equal(pthread_self(), currentThread))
   printf("The current thread ID matches my thread ID\n");
```

Synchronization and Critical Sections

- Race Condition
  - Threads are racing one another to change the same data structure.

Synchronization and Critical Sections

- Race Condition
  - Threads are racing one another to change the same data structure.
- Mutual Exclusion
  - Threads are not allowed to update shared data concurrently.
  - Accesses to shared data are encapsulated inside critical sections

Mutex Variables

- Mutex variables are one of the primary means of implementing thread synchronization and for protecting shared data when multiple writes occur.
- A mutex variable acts like a "lock" protecting access to a shared data resource.
- Only one thread can lock a mutex variable at any given time.

Example

Thread 1	Thread 2	Balance
Read balance: $1000		$1000
	Read balance: $1000	$1000
	Deposit $200	$1000
Deposit $200		$1000
Update balance $1000+$200		$1200
	Update balance $1000+$200	$1200

Mutex Variables

- A typical sequence in the use of a mutex is as follows:
  - Create and initialize a mutex variable
  - Several threads attempt to lock the mutex
  - Only one succeeds and that thread owns the mutex
  - The owner thread performs some set of actions
  - The owner unlocks the mutex
  - Another thread acquires the mutex and repeats the process
  - Finally the mutex is destroyed

Creating and Destroying Mutexes

- Mutex variables must be declared with type *pthread_mutex_t*
- It must be initialized before they can be used.
- There are two ways to initialize a mutex variable:
  1. pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;
  2. pthread_mutex_t mymutex ;
  3. pthread_mutex_init(&mymutex, NULL) ;
  4. pthread_mutex_destroy() should be used to free a
  - mutex object which is no longer needed.

Locking and Unlocking Mutexes

- **pthread_mutex_lock (&mymutex)**
  - It used by a thread to acquire a lock on the specified *mutex* variable. If the mutex is already locked by another thread, this call will block the calling thread until the mutex is unlocked.
- **pthread_mutex_unlock (&mymutex)**
  - It unlock a mutex if called by the owning thread. Calling this routine is required after a thread has completed its use of protected data if other threads are to acquire the mutex for their work with the protected data.

Sample Codes

1. dotprod_mutex.c

Mutex Deadlocks

- A deadlock occurs when one or more threads are stuck waiting for something that never will occur.



Example

mutex m1, m2;

Thread 1
Thread 2

```
lock(&m1);              lock(&m2);
//Do Something              //Do Something
lock(&m2);              lock(&m1);
//Do Something              //Do Something
        .                       .
        .                                                   .
        .                                                   .
unlock(&m2);                unlock(&m1);
//Do Something              //Do Something
unlock(&m1);                unlock(&m2);
```

Nonblocking Mutex

- **pthread_mutex_trylock (&mymutex)**
  - If you call *pthread_mutex_trylock* on an unlocked mutex, you will lock the mutex as if you had called *pthread_mutex_lock,* and *pthread_mutex_trylock* will return 0.
  - However, if the mutex is already locked by another thread, *pthread_mutex_trylock* will not block. Instead, it will return an error.
  - You may try again later to lock the mutex.

Semaphores for Threads

- A *semaphore* is a counter that can be used to synchronize multiple threads.
- A semaphore supports two basic operations:
  - **wait**
    - Decrements the value of the semaphore by 1.
    - If it is zero, the operations blocks until the value of the semaphore becomes positive.
  - **post**
    - Increments the value of the semaphore by 1.

Semaphores for Threads

- If you use semaphores, include *<semaphore.h>.*
- A semaphore is represented by *sem_t* variable.
  - Before using it, you must initialize it.

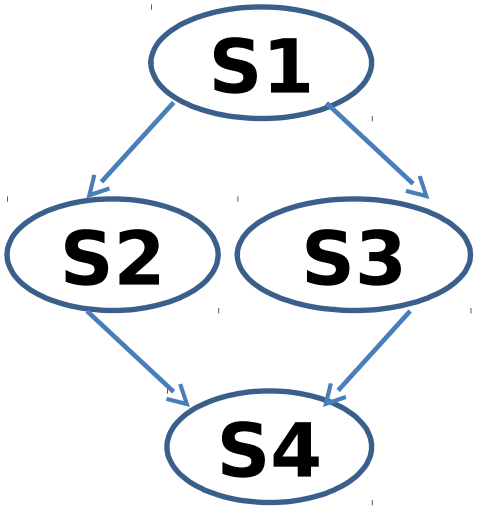
```
sem_t variable;
sem_init(variable, 0, init_value);
```

- To wait on a semaphore -> sem_wait
- To post to a semaphore -> sem_post
- To deallocate a semaphore -> sem_destroy

Sample Programs

1. mutex-example.c

Example



T1 -> S1, S4
T2 -> S2
T3 -> S3

Readers - Writers problem

- The problem deals with situations in which many threads must access the same shared memory at one time, some reading and some writing,
  - with the natural constraint that no thread may access the share for reading or writing while another process is in the act of writing to it.
- It is allowed for two or more readers to access the share at the same time.

Readers - writers problem

```
reader() {

}
```

```
writer() {

}
```

Readers - writers problem

```
reader() {

}
```

```
writer() {

    // Writing is done

}
```

Readers - writers problem

```
mutex writer;

reader() {

}
```

```
writer() {

    // Writing is done

}
```

Readers - writers problem

```
mutex writer;

reader() {

}
```

```
writer() {
  lock(writer);
    // Writing is
  done
  unlock(writer);
   }
```

Readers - writers problem

```
mutex writer;

reader() {

 // Do the Reading

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;

reader() {

    lock(writer);

 // Do the Reading

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;

reader() {

    lock(writer);

 // Do the Reading

    unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
```

```
reader() {

    lock(writer);

 // Do the Reading

    unlock(writer);

}
```

```
writer() {
   lock(writer);
    // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
```

```
reader() {

 readcount++;

     lock(writer);

 // Do the Reading



     unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
```

```
reader() {

 readcount++;
 if (readcount == 1)
     lock(writer);

 // Do the Reading



     unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
```

```
reader() {

 readcount++;
 if (readcount == 1)
     lock(writer);

 // Do the Reading

readcount--;

     unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
```

```
reader() {

 readcount++;
 if (readcount == 1)
      lock(writer);

 // Do the Reading

readcount--;
if (readcount == 0)
      unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
mutex reader;

  reader() {

   readcount++;
   if (readcount == 1)
        lock(writer);

   // Do the Reading

  readcount--;
  if (readcount == 0)
        unlock(writer);

  }
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
mutex reader;

reader() {
 lock(reader);
 readcount++;
 if (readcount == 1)
      lock(writer);

 // Do the Reading

readcount--;
if (readcount == 0)
      unlock(writer);

}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
mutex reader;

  reader() {
   lock(reader);
   readcount++;
   if (readcount == 1)
        lock(writer);
  unlock(reader);
   // Do the Reading

  readcount--;
  if (readcount == 0)
        unlock(writer);

  }
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
mutex reader;

  reader() {
   lock(reader);
   readcount++;
   if (readcount == 1)
        lock(writer);
  unlock(reader);
   // Do the Reading
  lock(reader);
  readcount--;
  if (readcount == 0)
        unlock(writer);

  }
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Readers - writers problem

```
mutex writer;
readcount=0;
mutex reader;
```

```
reader() {
 lock(reader);
 readcount++;
 if (readcount == 1)
      lock(writer);
unlock(reader);
 // Do the Reading
lock(reader);
readcount--;
if (readcount == 0)
      unlock(writer);
unlock(reader);
}
```

```
writer() {
   lock(writer);
     // Writing is
   done
   unlock(writer);
    }
```

Producer-Consumer Problem

- The problem describes two processes, the producer and the consumer, who share a common, fixed-size buffer used as a queue.
- The producer's job is to generate a piece of data, put it into the buffer and start again.
- At the same time, the consumer is consuming the data (i.e., removing it from the buffer) one piece at a time.
- The problem is to make sure that the producer won't try to add data into the buffer if it's full and that the consumer won't try to remove data from an empty buffer.

Producer-Consumer Problem

```
producer() {

}
```

```
consumer() {

}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();




}
```

```
consumer() {




}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();

    // Put item to the queue

}
```

```
consumer() {

}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();

    // Put item to the queue
    putItemIntoBuffer(item);

}
```

```
consumer() {

}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();

    // Put item to the queue
    putItemIntoBuffer(item);

}
```

```
consumer() {

    // Get item from the queue


}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();

    // Put item to the queue
    putItemIntoBuffer(item);

}
```

```
consumer() {

    // Get item from the queue
    item =
        removeItemFromBuffer()

}
```

Producer-Consumer Problem

```
producer() {
    item = produceItem();

    // Put item to the queue
    putItemIntoBuffer(item);

}
```

```
consumer() {

    // Get item from the queue
    item =
        removeItemFromBuffer();

    consumeItem(item);
}
```

Producer-Consumer Problem

```
mutex bufManip;
producer() {
    item = produceItem();

    // Put item to the queue
    putItemIntoBuffer(item);

}
```

```
consumer() {

    // Get item from the queue
    item =
        removeItemFromBuffer();

    consumeItem(item);
}
```

Producer-Consumer Problem

```
mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);


}
```

```
consumer() {


    // Get item from the queue
    item =
        removeItemFromBuffer();


    consumeItem(item);
}
```

Producer-Consumer Problem

```
mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);

}
```

```
consumer() {

    // Get item from the queue
    item =
        removeItemFromBuffer();

    consumeItem(item);
}
```

Producer-Consumer Problem

```
mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);

}
```

```
consumer() {

    lock(bufManip);
    // Get item from the queue
    item =
        removeItemFromBuffer();


    consumeItem(item);
}
```

Producer-Consumer Problem

```
mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);

}
```

```
consumer() {

    lock(bufManip);
    // Get item from the queue
    item =
        removeItemFromBuffer();
    unlock(bufManip);

    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items

mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);

}
```

```
consumer() {

    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);

    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items

mutex bufManip;
```

```
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);

}
```

```
consumer() {
    sem_wait(fullCount);
    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);

    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items

mutex bufManip;
producer() {
    item = produceItem();

    lock(bufManip);
    // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);
    sem_post(fullCount );
}
```

```
consumer() {
    sem_wait(fullCount);
    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);

    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items

mutex bufManip;
producer() {
    item = produceItem();
    sem_wait(emptyCount );
    lock(bufManip);
     // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);
    sem_post(fullCount );
}
```

```
consumer() {
    sem_wait(fullCount);
    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);

    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items

mutex bufManip;
producer() {
    item = produceItem();
    sem_wait(emptyCount );
    lock(bufManip);
     // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);
    sem_post(fullCount );
}
```

```
consumer() {
    sem_wait(fullCount);
    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);
    sem_post(emptyCount );
    consumeItem(item);
}
```

Producer-Consumer Problem

```
semaphore fullCount = 0; //  # of items
semaphore emptyCount = BUFFER_SIZE  //  remaining
space
mutex bufManip;
```

```
producer() {
    item = produceItem();
    sem_wait(emptyCount );
    lock(bufManip);
     // Put item to the queue
    putItemIntoBuffer(item);
    unlock(bufManip);
    sem_post(fullCount );
}
```

```
consumer() {
    sem_wait(fullCount);
    lock(bufManip);
    // Get item from the queue
    Item =
        removeItemFromBuffer();
    unlock(bufManip);
    sem_post(emptyCount );
    consumeItem(item);
}
```